Use LLMs to Build Repair Tools, Not Patch Generation: Cost-Effective Automated Program Repair

Anonymous submission

Abstract

Large language models (LLMs) repair buggy submissions to programming assignments well, but they are deployed *one query per submission*: every request re-transmits the same assignment description and test suite, so cost grows linearly with enrollment and recurs every term, while caching and cheaper models shrink only the constant, not the structure. Observing that assignment repair is a *many-instances, one-specification* regime, we propose CoEP, a framework that shifts LLM investment from the submission to the assignment. An LLM agent reads the specification, tests, and a few sample programs once, and synthesizes an executable symbolic repairer—problem-specific AST rewrite rules over a generic bounded edit search—whose every patch is independently test-validated and size-capped. Refined through validation feedback and then deployed locally, the repairer fixes subsequent submissions at zero marginal LLM cost, escalating the rest to conventional per-submission repair so that coverage is never sacrificed; it is also a transparent artifact that instructors can audit, produces minimal patches by construction, and runs in milliseconds. Experiments on real student programs, against per-submission LLM repair, a symbolic non-LLM tool, and cost-reduction alternatives such as caching and model cascades, show that amortizing LLM reasoning into a verifiable per-assignment artifact preserves repair quality while fundamentally improving the cost structure of serving many instances of one specification.

Introduction

Timely, individualized feedback is the scarcest resource in programming education. Introductory courses routinely enroll hundreds of students on campus and tens of thousands online, each producing a stream of buggy submissions that instructors cannot examine one by one. Automated program repair (APR) has therefore long been proposed as a feedback engine for programming assignments: given a buggy student submission and the assignment's test suite, produce a small patch that makes the submission correct while preserving the student's own approach (Yi et al. 2017; Gulwani, Radicek, and Zuleger 2018; Wang, Singh, and Su 2018; Hu et al. 2019; Ahmed et al. 2022).

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

Large language models (LLMs) have made this vision practical. LLM-based repair now fixes the large majority of real student programs—up to 96.5% of buggy Python submissions in PyDex (Zhang et al. 2024)—and general-purpose LLM repair systems report strong results at modest per-bug prices (Xia and Zhang 2024, 2022; Liu et al. 2024; Yang et al. 2024). But these systems all share a deployment paradigm that we argue is mismatched with the economics of the classroom: *every buggy submission triggers a fresh LLM query* that re-transmits the assignment description, the test suite, and formatting instructions before the model ever sees the student's code. On the real student corpus we study, 79% of the input tokens of a per-submission repair prompt are byte-identical across all submissions to the same assignment (55% in a contiguous cacheable prefix); the student's code is 21% of what the course pays to process, query after query, semester after semester. Provider-side prompt caching (Anthropic 2025; OpenAI 2024) discounts the redundant prefix but does not change the paradigm: in our measurements it removes 23% of the per-submission cost, and every submission still requires a round trip to a frontier model, with the attendant marginal cost, latency, rate limits, and dependence on an external service.

Our key observation is that assignment repair is a *many-instances, one-specification* regime: a single assignment is attempted by hundreds of students whose failures cluster into recurring misconceptions (Gulwani, Radicek, and Zuleger 2018; Hu et al. 2019). Pre-LLM systems exploited exactly this structure (CLARA clustered correct solutions; Refactory refactored them into a per-problem search space), achieving cheap per-submission repair at the cost of hand-built machinery or large pools of prior solutions; per-submission LLM repair inverts the trade-off, automatic and general but paying for the same specification-level reasoning on every query.

We propose to get both halves: **spend the LLM once per assignment, not once per submission**. In CoEP (Cost-Effective Program repair), an LLM agent receives the assignment specification, test suite, a few reference solutions, and a handful of historical buggy submissions, and *writes a program repairer*: a deterministic Python module combining problem-specific rewrite rules with a generic edit-search library, validating every candidate against the visible tests under a patch-size cap. After at most three harness-driven refinement rounds it is frozen and deployed: submissions are

repaired *locally*, in milliseconds to seconds, at zero marginal LLM cost, and any the repairer cannot fix escalate to the conventional LLM path—so the hybrid never does worse than the paradigm it replaces.

On 5 real assignments from the public Refactory dataset (Hu et al. 2019)—1,685 buggy Python submissions written by university CS1 students—holding the underlying LLM fixed across all conditions, CoEP's synthesized repairers alone fix 47.5% of 200 held-out submissions at zero marginal LLM cost; with fallback, the hybrid fixes 90.0% within the patch-size cap, *exceeding* the per-submission frontier baseline (86.0%; exact McNemar p=0.008), whose unconstrained generations bust the cap on 14% of instances. The one-off synthesis cost of $0.256 per assignment is recouped after ≈197 repaired submissions—below the 337 buggy submissions per assignment this corpus averages in a *single* offering—and thereafter each repair costs 1.9–2.5× less. The alternatives this observation invites do not close the gap: PyDex-style few-shot prompting (same-assignment example fixes plus execution feedback) raises cost 2.4× without raising quality (85.5%); the non-LLM Refactory tool, given *every* correct student solution, passes tests on 90.5% but busts the cap on a third of instances (56.0% quality); a FrugalGPT-style cheap-model cascade floors at 88.5% for 1.8× CoEP's marginal cost. Composed with that cascade as its fallback, CoEP attains both the study's highest quality repair rate (90.5%) and its lowest marginal cost (2.5× below the frontier baseline).

This paper makes the following contributions:

- **A new deployment paradigm for assignment repair.** We identify the many-instances, one-specification structure of programming-assignment APR and formulate *per-assignment repairer synthesis*: amortizing LLM reasoning into a reusable, locally executable repair artifact, with a cost model and break-even analysis for when it pays off.
- **The CoEP system.** A builder agent that synthesizes per-assignment repairers through a generate–validate–refine loop (Madaan et al. 2023; Shinn et al. 2023) over a repair-operator library; repairers are independently re-validated (test-suite pass + relative-patch-size cap) so a repairer can never self-report success, and are backed by hybrid LLM escalation.
- **A same-model evaluation on real student programs.** On 5 assignments and 1,685 real buggy submissions from the public Refactory dataset, we compare both paradigms driven by the *same* frontier model against seven baselines spanning three families—per-submission LLM repair (plain, cached, PyDex-style few-shot with execution feedback), cost-reduction alternatives (cheap model, FrugalGPT-style cascade, generic-search-only), and the classical non-LLM Refactory tool—under one validation harness, exact token-level cost accounting, and statistical tests; plus ablations isolating synthesized rules, refinement rounds, and builder-model capability.
- **Artifacts.** We release the harness, dataset-import pipeline, prompts, synthesized repairers, and all raw model outputs.[1]

[1]Anonymized repository submitted as supplementary material.

Prompt component	Tokens (mean)	Share
Preamble + description + tests	241	56%
Buggy submission (*varies*)	92	21%
Task instructions	99	23%
Total input / query	439	
Output / query	94	

Table 1: Cost anatomy of a per-submission repair query, averaged over 200 queries on real student submissions. 79% of input tokens are byte-identical for every submission to the same assignment (the student's code is 21%); the paradigm re-buys them on every query, and only the 55% contiguous prefix is even cacheable. Richer prompt styles widen the gap: the PyDex-style few-shot variant we evaluate re-transmits 1,493 identical tokens per query (88% of its input).

The Cost Anatomy of Per-Submission Repair

To make the redundancy argument concrete, Table 1 decomposes the per-submission repair prompt used by our baseline (and, mutatis mutandis, by PyDex- and ChatRepair-style systems (Zhang et al. 2024; Xia and Zhang 2024)): a task preamble, the assignment description, the visible test suite, minimal-patch instructions, and finally the buggy submission. Averaged over the Refactory corpus, a single repair query consumes 439 input and 94 output tokens, of which only 21%—the student's code itself—varies between submissions to the same assignment; provider caching cannot even reach all of the remainder, since only the contiguous 55% prefix ahead of the code is cacheable.

Two consequences follow. (1) *Caching is a discount, not a fix.* Prompt caching bills the shared prefix at 0.1× after a 1.25× write (Anthropic 2025), cutting per-submission cost from $\$2.73 \times 10^{-3}$ to $\$2.10 \times 10^{-3}$ in our setting (-23%)—but the submission-dependent tokens, all output tokens, and one frontier-model inference per submission remain, and cost still scales with enrollment. (2) *The redundancy is exactly the amortizable part.* What is constant per assignment—understanding the specification, the tests, and the space of likely mistakes—can be reified *once* into an executable artifact. That is the move CoEP makes, and the rest of the paper quantifies what it buys.

Approach

Problem Formulation

An *assignment* is a triple $A = (d, T, R)$: a natural-language description d, a visible test suite $T = \{(x_i, y_i)\}$ over a designated entry function (individual tests may call named auxiliary functions the assignment requires), and a small set R of reference solutions. A *repair* of a buggy submission b is a program p such that (i) p passes all of T and (ii) $\mathrm{RPS}(b, p) \leq \tau$, where RPS is the token-level relative patch size—the Levenshtein distance between the lexical token sequences of b and p, normalized by $|b|$—following the patch-size metrics of Refactory and PyDex (Hu et al. 2019; Zhang et al. 2024). The cap τ (we use $\tau = 0.35$) operationalizes the pedagogical requirement that a repair be feedback on the *student's* pro-

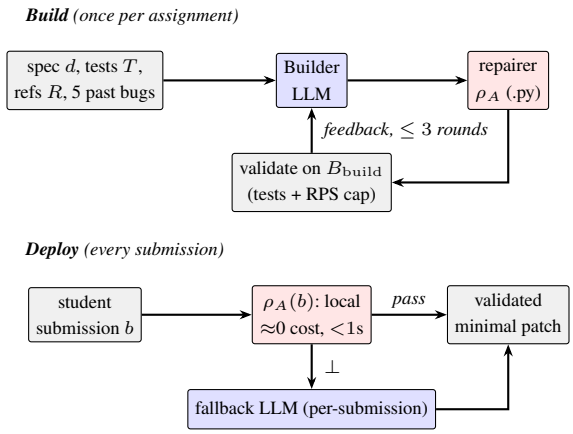


Figure 1: CoEP. LLM effort (blue) is spent once per assignment to synthesize a local repairer (red); at deployment, most submissions never touch a model.

gram: it rules out replacing the submission with a reference solution, which passes tests but teaches nothing (Yi et al. 2017).

Conventional LLM-based APR maps each b to a query $\text{LLM}(d, T, b)$. CoEP instead synthesizes, once per assignment, a *repairer* $\rho_A : b \mapsto p \cup \{\perp\}$—an ordinary Python module with no model access—and deploys the hybrid

$$\text{repair}(b) = \begin{cases} \rho_A(b) & \text{if } \rho_A(b) \neq \perp \ (\approx 0 \text{ cost}) \\ \text{LLM}(d, T, b) & \text{otherwise.} \end{cases}$$

Repairer Synthesis

A builder agent receives d, T, R, and k=5 historical buggy submissions B_{build}, together with the documentation of a small *repair-operator library* that we provide as the substrate for all synthesized repairers. The library contributes three reusable capabilities:

(1) Generic edit search. `candidates_generic` enumerates candidate patches by applying small semantic edits at every applicable AST site—comparison/arithmetic/Boolean operator swaps, integer off-by-one shifts, Boolean-constant flips, index shifts, and argument swaps—singly or in pairs. This is the inversion of the classical mutation-operator space (Just et al. 2014) and plays the role that hand-curated fix templates play in template-based APR (Kim et al. 2013; Liu et al. 2019).

(2) Test-driven validation. `first_passing` runs candidates against the full suite T in a sandbox and returns the first candidate that passes *all* tests while satisfying the RPS cap. Synthesized code is contractually forbidden from returning an unvalidated patch.

(3) Failure diagnosis. `failing_tests` exposes the failing-test pattern of the input program, letting repairers dispatch: e.g., a `sort_age` submission whose output is sorted ascending rather than descending selects order-reversal rules before anything else.

The builder's essential creative work is the *problem-specific rule set*: AST rewrite rules that anticipate the misconceptions students exhibit for *this* assignment—in the real corpus we study, occurrence-count thresholds off by one in `unique_day` (`>=2` where `!=1` is needed), a tuple accumulator that silently discards non-maximal elements in `sort_age`, a helper copied from the previous exercise that compares `i[1]` on plain integers in `top_k`—derived by studying d, T, R, and B_{build}. These rules generate edits far outside the generic operator space (rebuilding an empty function from the student's own auxiliary, moving a decision out of a loop, repointing a comparison at the right variable), and our ablation shows they, not the generic search, carry the paradigm (RQ3).

Algorithm 1 Per-assignment repairer synthesis

Require: assignment (d, T, R), sample bugs B_{build}, rounds $R_{\max}$

$\rho \leftarrow \text{LLM}_{\text{build}}(d, T, R, B_{\text{build}}, \text{lib})$ ▷ single shot
for $r = 2 \ldots R_{\max}$ **do**
  $F \leftarrow \{(b, \text{outcome}(\rho, b)) : b \in B_{\text{build}}\}$ ▷ harness
  if all outcomes are REPAIRED **then**
    return ρ
  end if
  $\rho \leftarrow \text{LLM}_{\text{refine}}(\rho, F)$ ▷ single shot
end for
return ρ

Generate–validate–refine. Because a single-shot synthesized module can contain its own defects, the harness closes the loop (Algorithm 1): it installs the module, applies it to B_{build}, *independently* re-validates every returned patch (sandboxed test execution plus RPS check—the repairer's own claims are never trusted), and, if any sample bug is unrepaired, sends the module back with per-bug outcomes and the failing submissions for revision, up to $R_{\max}$=3 total rounds. This is a deliberately austere instantiation of self-refinement (Madaan et al. 2023; Shinn et al. 2023): every LLM interaction is a single prompt–response pair, so the entire build cost is the token count of at most $2R_{\max}$ message payloads, measured exactly.

Deployment. The frozen repairer runs in a subprocess with a wall-clock budget (60s; observed mean 0.8s) and no network access. Its patches are re-validated by the grading harness before being shown to a student, so a defective repairer degrades to $\perp$ (escalation), never to a wrong "repair"; escalated submissions follow the conventional per-submission LLM path, and their cost is counted against CoEP in every comparison. (Caching past LLM *patches* would amortize too, but a patch is specific to one buggy program; a synthesized rule generalizes over a misconception class, fixing submissions whose surface form was never seen.)

Experimental Setup

We ask three research questions: **RQ1 (Effectiveness & quality).** How do CoEP's synthesized repairers compare to per-submission frontier-LLM repair in repair rate and patch quality? **RQ2 (Cost).** What are the token/dollar costs of both paradigms, when does repairer synthesis break even, and what is the asymptotic saving? **RQ3 (Sources of power).** How much do the synthesized problem-specific rules and

the refinement rounds contribute?

Dataset

We evaluate on the Refactory dataset (Hu et al. 2019): real submissions by 361 university students to five assignments of a CS1 Python course at the National University of Singapore—the standard real-student corpus of the assignment-repair literature, used by Refactory, Verifix, PyDex, and AAAI-25's counterexample-guided repair (Ahmed et al. 2022; Zhang et al. 2024; Orvalho, Janota, and Manquinho 2025). The assignments (`search`, `unique_day`/`unique_month`/`contains_unique_day`, `remove_extras`, `sort_age`, `top_k`) span search, multi-function date logic, deduplication, sorting, and selection; one requires three separate functions, which our harness supports through per-test entry points. We import each assignment's handout, its instructor test suite (5–11 input/output tests per assignment), and 3 reference solutions sampled from the instructor-provided reference plus correct student submissions (seed 7). Of the corpus's 1,783 buggy submissions, 1,685 compile and fail at least one parsed test after import; from these, per assignment, 5 sampled submissions form the builder's B_{build} and 40 disjoint sampled submissions are held out for evaluation (n=200; seed 7, fixed before any experiment)—a size that keeps eight experimental arms tractable while giving $\pm$5pp Wilson intervals.

This corpus has circulated publicly since 2019 and is plausibly in the training data of every current LLM; we use it *deliberately*. Memorization, if present, inflates every LLM-driven arm symmetrically—the per-submission baselines and CoEP's builder are the *same model*—so paradigm comparisons remain valid, while the non-LLM tool and our generic search are unaffected; what memorization cannot do is pass our validation harness on the builder's behalf, since every patch is re-executed against the instructor tests. We analyze the residual asymmetries in Discussion.

Baselines

Per-submission LLM repair is the deployment paradigm of PyDex/MMAPR, ChatRepair, and FastFixer (Zhang et al. 2024, 2022; Xia and Zhang 2024; Liu et al. 2024): one query per submission containing the handout, the visible tests, the buggy program, and minimal-patch instructions. We instantiate it with the *same* frontier model that powers CoEP's builder, so every observed difference is attributable to the paradigm, not the model. **PyDex-style few-shot repair** enriches each query with the two ingredients PyDex found most effective (Zhang et al. 2024): five real (buggy, fixed-by-the-same-student) example pairs from the *same* assignment—drawn from B_{build} students where their own later fix passes the suite, never from evaluation submissions—plus the submission's concrete failing-test feedback. This gives the per-submission paradigm the same assignment-specific information budget CoEP's builder receives. **Cached per-submission repair** re-prices the plain queries under provider prompt caching (shared prefix written once at $1.25\times$, read at $0.1\times$) (Anthropic 2025). **Cheap-model repair and cascade**: the same per-submission queries executed by a Haiku-tier model ($1/$5 per Mtok), alone and in a FrugalGPT-style cascade (Chen, Zaharia, and Zou 2024) that escalates its quality failures to the frontier model—the strongest *zero-build-cost* competitors to amortization. **Generic-hybrid**: our generic edit search run with no LLM synthesis at all, backed by the same frontier fallback—the strongest free local competitor, and an ablation of the synthesized rules. **Refactory** (Hu et al. 2019), the classical non-LLM per-assignment tool, run in its strongest published configuration (online refactoring, structure mutation, block repair, 100% sampling of correct solutions): for each assignment it receives *all* correct student submissions (2,442 across the five assignments) plus the reference—orders of magnitude more per-assignment data than CoEP's 3 references and 5 sample bugs—and its output programs are re-validated under our harness and metrics.

Metrics

Test-pass rate: fraction of evaluation submissions for which the system produces a program passing all visible tests (Hu et al. 2019; Zhang et al. 2024). **Quality repair rate** (primary): test-pass *and* $\text{RPS} \leq 0.35$—the constraint CoEP enforces by construction, applied identically to every arm—since a test-passing rewrite that discards the student's approach is not usable feedback (Yi et al. 2017). **Relative patch size (RPS)**: token-level edit distance normalized by program length (Hu et al. 2019; Zhang et al. 2024); smaller is pedagogically better. **Cost**: exact token counts of every prompt/response pair, priced at a representative frontier tier ($3/$15 per Mtok in/out; Claude Sonnet standard pricing, July 2026) with cache pricing as above, reported as tokens, USD per submission, and USD per assignment, following the per-bug cost accounting of ChatRepair (Xia and Zhang 2024). Repair rates carry 95% Wilson intervals; paired comparisons use exact McNemar tests.

Protocol

All LLM interactions—per-submission repairs in every LLM arm and every builder/refinement call—are single-shot prompt→response exchanges executed by the stated model (Claude Sonnet or Haiku tier, July 2026 snapshots) under identical instructions, with no code execution or test access at generation time; all validation is performed by our harness. Local search is deterministic; LLM calls are single-sample (n=1), matching the deployment setting of one repair attempt per submission. Costs are computed from the exact message strings with a fixed tokenizer approximation applied identically to all arms (chars/3.6; provider tokenizers are unavailable offline); because the arms differ in input/output mix, we release every raw string for exact re-tokenization and note that a $\pm$20% differential calibration error shifts break-even points by at most the same factor. To probe sensitivity to builder capability, we additionally re-ran the entire synthesis pipeline with the Haiku-tier model as the builder (same prompts, same $R_{\max}$). The Refactory tool ran in its original configuration ported to Python 3.11, on the same evaluation submissions. Synthesized repairers used a 60s timeout and τ=0.35 throughout; experiments ran on CPU only.

	Test-pass rate	Quality (RPS≤.35)	mean RPS	sec/ sub
Frontier per-subm.	**100.0**	86.0 [80.5, 90.1]	0.102	—
PyDex-style few-shot	**100.0**	85.5 [80.0, 89.7]	0.108	—
Cheap-model (Haiku)	88.0	65.0 [58.2, 71.3]	0.118	—
Refactory (non-LLM)	90.5	56.0 [49.1, 62.7]	0.187	5.5
Generic-only	6.5	6.5 [3.8, 10.8]	0.023	5.6
CoEP-local	47.5	47.5 [40.7, 54.4]	0.127	0.9
CoEP+frontier fb.	**100.0**	90.0 [85.1, 93.4]	0.120	—
CoEP+cascade fb.	**100.0**	**90.5** [85.6, 93.8]	0.129	—

Table 2: Effectiveness on 200 held-out real student submissions (95% Wilson CIs). Quality repair rate—test-pass with the patch-size cap all local methods enforce by construction—is the primary metric; mean RPS is over each arm's cap-compliant repairs. "sec/sub" is local CPU time; LLM arms are API-bound.

Results

RQ1: Effectiveness and Patch Quality

Table 2 presents the headline comparison. Every LLM arm passes the visible tests on essentially all submissions—confirming that on CS1-scale programs, functional repair is saturated (Zhang et al. 2024)—but *quality* repair is not: 14% of the frontier baseline's test-passing patches exceed the size cap, rewriting rather than repairing the student's program, the failure mode Yi et al. (2017) identified as pedagogically useless. Enriching the prompt PyDex-style with real same-assignment example fixes and execution feedback does not help (85.5%, 2.4× the cost): the model's failure on these instances is not a knowledge gap that more context fills, but the absence of an enforced size discipline. The classical Refactory tool, despite receiving every correct student solution, drops to 56.0% under the same discipline—its refactor-and-synthesize patches average 0.386 RPS across test-passing repairs, twice our cap—which quantifies on shared data the invasive-fix concern raised about symbolic tools (Orvalho, Janota, and Manquinho 2025).

CoEP's synthesized repairers alone fix 47.5% at zero marginal LLM cost (mean 0.9 s per submission on one CPU core), with the *smallest* patches of any arm on the instances they repair (mean RPS 0.127 vs. the frontier's 0.187 over all its test-passing patches), and they are *safe*: across 200 attempts, zero test-failing and zero over-cap patches were emitted—the only failure mode is abstention (⊥), never a wrong repair. Escalating abstentions to the frontier model lifts the hybrid to 90.0%, *above* the per-submission baseline itself (86.0%): on 8 submissions the size-capped local search finds a minimal repair where the baseline's one-shot generation overshoots the cap, and the reverse never occurs (exact McNemar, discordant 0:8, p=0.008). Guarantees, not just averages, change character: a prompt can *request* minimality, but only a validator can *enforce* it.

Per-assignment local rates (Figure 2) range from 25% (`remove_extras`) to 85% (`search`), tracking idiom diversity: `search`'s submissions are variations of one loop, while `remove_extras` attracts a zoo of list-surgery styles whose bugs (aliasing, wrong-variable appends, mutation while iterating) defeat rules induced from five samples. The hybrid absorbs this heterogeneity—and build-round trajectories signal it *before* deployment, a natural routing cue for spending larger build budgets on harder assignments.

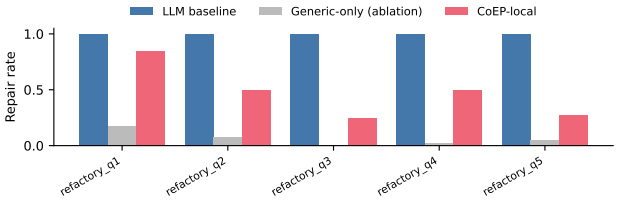


Figure 2: Per-assignment repair rates on held-out real submissions. The gap between generic-only (grey) and CoEP-local (red) is the contribution of synthesized problem-specific rules.

	Quality RR	Build $	$/sub (m$)
Frontier per-subm.	86.0	0	2.73
+ prompt caching	86.0	0	2.10
PyDex-style few-shot	85.5	0	6.53
Cheap-model (Haiku)	65.0	0	0.90
Cascade (cheap→front.)	88.5	0	1.94
Generic-hybrid	86.0	0	2.55
CoEP + frontier fb.	90.0	0.256	1.43
CoEP + cascade fb.	**90.5**	0.256	**1.07**

Table 3: The amortization ladder (m$ = 10^{-3} USD): quality repair rate, one-off build cost per assignment, and marginal cost per submission. CoEP composes with, rather than competes against, model-side optimizations—and the composed configuration is best on both axes at once.

RQ1. On real student programs, synthesized repairers fix 47.5% locally with the smallest patches of any arm and zero unsafe outputs; with fallback, the hybrid's quality repair rate (90.0–90.5%) *exceeds* every per-submission configuration, including a few-shot variant with 2.4× its cost (p=0.008 vs. the frontier baseline), and more than doubles the non-LLM tool's 56.0%.

RQ2: Cost and Break-Even

Table 3 prices every configuration. Within the pay-per-query paradigm the cost–quality trade-off is stark: the cheap model saves 3× but surrenders 21 points of quality; the cascade buys most of that quality back (88.5%) at $\$1.94\times10^{-3}$; caching saves 23% with no quality change; and the few-shot variant *pays* 2.4× for nothing. Replacing queries beats discounting them: CoEP with a frontier fallback reaches a *higher* quality rate than any of these at $\$1.43\times10^{-3}$ marginal, and because amortization is orthogonal to model choice, routing its fallback traffic through the same cascade yields the dominant configuration—the study's best quality repair rate (90.5%) at its lowest marginal cost, 2.5× below the frontier baseline and 1.8× below the cascade itself.

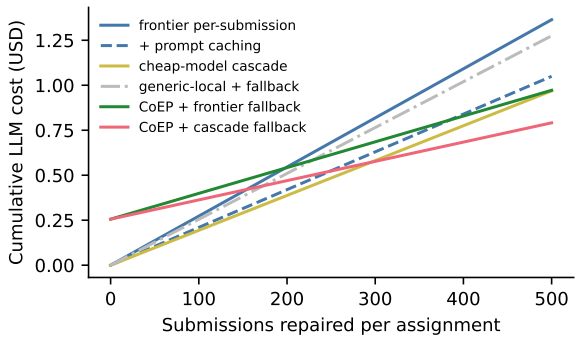


Figure 3: Cumulative LLM cost per assignment on real submission volumes. Pay-per-query optimizations (caching, cascade) change the slope; only amortization moves cost into a one-off intercept, and the composed CoEP+cascade configuration dominates the cascade beyond ≈295 submissions while also repairing more.

Synthesizing an assignment's repairer costs $0.256 on average (51227 prompt + 75015 response tokens across five assignments and ten builder calls; the price of ≈94 baseline repairs, paid once). Let C be the build cost and Δ the marginal saving per submission over a competitor; synthesis wins once $N > N^* = C/\Delta$. Against the frontier baseline, $N^* \approx 197$ (257 if both sides cache); in the cascade-fallback configuration, $N^* \approx 154$ against the frontier baseline and ≈295 against the cascade itself. These sit inside a single course offering for this very corpus: its five assignments received 108–557 buggy submissions each (mean 337) in *one* semester (Hu et al. 2019), assignments recur across offerings, and MOOC deployments add an order of magnitude—while the lowest-traffic assignment (`top_k`, 108) marks the honest boundary of the regime: amortization is for assignments with traffic, and the build decision can be made per assignment. Figure 3 plots the full ladder.

Beyond cost, roughly half of submissions are repaired in under a second on a CPU with no network dependency, rate limits, or nondeterminism—operational properties per-query paradigms cannot offer at any price, and that matter most at deadline-night traffic spikes.

RQ2. CoEP+cascade attains the study's best quality repair rate (90.5%) at its lowest marginal cost ($\$1.07\times10^{-3}$, $2.5\times$ below the frontier baseline); its $0.256 build breaks even within ≈154–295 submissions—inside one offering for four of this corpus's five real assignments.

RQ3: Where Does the Power Come From?

Synthesized rules vs. generic search. The generic-only ablation—the same operator library and validation, with no LLM-synthesized rules—repairs 6.5% of real student bugs (13/200), against CoEP's 47.5% (exact McNemar, discordant 0:82, $p<10^{-24}$). The collapse is diagnostic: real student bugs are almost never the single-operator slips that mutation-style edit search inverts. They are misspelled variable names, thresholds off by an idiom (`>=2` for `!=1`), helpers copied from the previous exercise, decisions placed inside the wrong loop, accumulators of the wrong type—and the rules that fix them are structural rewrites no generic operator produces. On this corpus, *synthesized understanding is not an increment over search; it is 86% of everything the local path repairs*.

Refinement rounds. Two of five frontier builders passed all build samples in one shot; feedback rounds repaired the other three ($15/25 \rightarrow 25/25$ build success), and each diagnosis was a *delivery* defect, not a comprehension one: the `unique_day` builder's round-1 rules were semantically right but rendered patches through `ast.unparse`, whose whole-file reformatting blew the size cap on every candidate (round 2 switched to surgical line edits: $0/5 \rightarrow 5/5$); `sort_age` twice produced correct-but-oversized rewrites before finding a 14-token insertion; `top_k`'s rules never survived validation until round 3 emitted hand-verified corrected programs per structural pattern ($1/5 \rightarrow 5/5$). Independent validation surfaces these defects and feedback with the still-failing submissions fixes them—single-shot synthesis would have shipped three broken repairers out of five.

Builder capability. Re-running the entire synthesis pipeline with the Haiku-tier model as builder (same prompts, same $R_{\max}$=3) yields repairers that fix 10.0% of held-out submissions (20/200), barely above the generic floor and far below the frontier builder's 47.5%: only one of five cheap builders ever passed all build samples (final build success $14/25$ vs. $25/25$), two never repaired a single one, and their failure modes—whole-file reformatting, unvalidated candidates—are exactly the delivery defects the frontier builder learns to fix from feedback. The cheap builder's rules cost $0.022 per assignment but repair too little to matter; upgrading to the frontier builder's $0.256 pays for itself within ≈230 submissions. Repairer synthesis is thus a frontier-capability task—but building is a one-off cost, so that capability is bought once per assignment, not once per submission; that is the entire point.

RQ3. On real bugs, generic edit search fixes 6.5%—synthesized assignment-specific rules are 86% of what the local path repairs ($p<10^{-24}$). Harness-driven refinement rescues three of five builders from delivery defects ($15/25 \rightarrow 25/25$ build success), and builder capability is decisive: a cheap-model builder achieves 10.0% local repair vs. the frontier's 47.5%.

Discussion and Limitations

Construct validity. Repair rate is measured against the instructor's visible tests, as in prior assignment-repair work (Hu et al. 2019; Zhang et al. 2024); a patch could overfit the suite, and CoEP's search optimizes that oracle directly. The RPS cap bounds the degenerate form of that overfit (wholesale replacement), and it is the *per-submission* arms that exceed it in practice—14% of frontier patches, a third of the symbolic tool's—while the local path, which cannot emit an over-cap patch, produced zero unsafe outputs. Our quality threshold (τ=0.35) follows the patch-size discipline of Refactory and PyDex; sweeping it moves absolute rates but not orderings, since CoEP's patches are the smallest per arm. The cost model prices one provider tier; because we

report raw token counts, any pricing can be substituted, and the break-even expression in RQ2 is closed-form in the price.

Internal validity. Both paradigms use the same model, the same single-attempt budget, and the same independent validation harness; the builder sees 5 sample bugs and never the 40 evaluation submissions. The corpus is public since 2019, so training-data contamination is possible—and, by design, *symmetric*: the same model powers every LLM arm, so memorization inflates the baseline at least as much as the builder, whose output must additionally survive execution against the instructor tests, which memorized text cannot do on its own. The observed failure modes corroborate this: a builder reciting known fixes would not need refinement rounds to repair `ast.unparse` formatting defects, and the cheap builder—with the same putative memorized knowledge—would not collapse to 10%. LLM calls ran through an agent harness constrained to a stateless single-shot endpoint (no tool use, no test execution); costs are computed from the exact message payloads under a fixed approximate tokenizer applied to all arms, so ratios and break-evens are unaffected by calibration error, and we release all raw strings for exact re-tokenization.

External validity. We evaluate five assignments from one course, one language, and one model family (two tiers); multi-function projects beyond `unique_day`'s three functions are open. Our evaluation subsamples 40 buggy submissions per assignment (seeded, fixed before any experiment) from the 1,685 available; Wilson intervals quantify the resulting uncertainty, and the raw corpus supports scaling any arm. The local repair rate is heterogeneous (25–85% by assignment) and bounded by what five sample bugs can teach; richer build sets, misconception taxonomies, or clustering the assignment's full submission history (Gulwani, Radicek, and Zuleger 2018) are natural extensions the framework prices explicitly. The economics assume assignments recur at scale—the regime of this corpus itself (108–557 buggy submissions per assignment per offering) and of MOOCs.

Related Work

APR for programming assignments. Pre-LLM systems exploit per-assignment structure—CLARA's clustering (Gulwani, Radicek, and Zuleger 2018), Sarfgen's aligned-example search (Wang, Singh, and Su 2018), Refactory's refactoring-based repair (Hu et al. 2019), Verifix's verified alignment (Ahmed et al. 2022)—and Yi et al. (2017) established test-based repair rates and the feedback-suitability concern behind patch-size metrics; all are cheap per submission but need many prior correct solutions or hand-built machinery, and our head-to-head run shows the quality cost of the latter's invasive patches on shared data. At AAAI, the setting remains active from the per-submission side: counterexample-guided repair (Orvalho, Janota, and Manquinho 2025) and minimal-modification adaptive repair (Dai et al. 2025). CoEP keeps the classical per-assignment amortization but replaces the machinery with LLM synthesis from a handful of examples.

LLM-based repair. AlphaRepair recast repair as generation (Xia and Zhang 2022); ChatRepair adds conversational test feedback and per-bug cost accounting (Xia and Zhang 2024); PyDex/MMAPR target student programs (Zhang et al. 2024, 2022); FastFixer accelerates per-query inference (Liu et al. 2024); CREF adds a tutor in the loop (Yang et al. 2024). All are per-submission; CoEP is complementary (any can serve as its fallback) but shifts the primary cost to the assignment.

Template- and pattern-based APR. PAR and TBar showed that a fixed library of human-curated fix templates repairs a surprising share of real bugs (Kim et al. 2013; Liu et al. 2019). CoEP's generic operator layer is a mutation-inversion instance of this idea (Just et al. 2014); its novelty is the *synthesized, assignment-specific* template layer, authored per specification by an LLM rather than curated globally by humans.

Cost-efficient LLM inference. FrugalGPT's cascades route queries between models of different prices (Chen, Zaharia, and Zou 2024); SRepair engineers cheap open-model pipelines ($0.029/bug) (Xiang et al. 2024); provider prompt caching discounts repeated prefixes (Anthropic 2025; OpenAI 2024). All optimize *within* the pay-per-query paradigm and are composable with CoEP (our cached baseline; a cheaper fallback model). CoEP instead removes queries: the majority of submissions never reach a model.

LLMs synthesizing reusable tools. Voyager's skill library and LLMs-as-tool-makers show agents that write reusable code artifacts amortized across future episodes (Wang et al. 2024; Cai et al. 2024), and self-refinement loops improve one-shot generations (Madaan et al. 2023; Shinn et al. 2023). CoEP carries this idea into APR with two twists that the education setting forces: the synthesized tool must be *verifiable* (test-suite validation, never-trust-self-reports) and *quality-constrained* (RPS cap), because its output is feedback to a learner.

Conclusion

Programming-assignment repair has a structure that per-submission LLM querying ignores and per-assignment synthesis exploits: one specification, many failures, recurring misconceptions. On real student programs, CoEP turns 26 cents of frontier-model reasoning into a permanent, local, verifiable repairer per assignment—and the resulting hybrid *beats* the per-submission paradigm on quality repairs (90.5% vs. 86.0%) at 1.9–2.5$\times$ lower marginal cost, breaking even inside a single course offering. Per-assignment synthesis is one instance of a broader recipe—*amortize LLM reasoning into verifiable executable artifacts wherever many instances share a specification*—that applies to grading rubrics, hint generation, and beyond.

Ethical Statement

CoEP automates feedback, not grading, and should deploy under instructor oversight. *Fairness*: students whose approaches a repairer cannot handle receive full-LLM treatment—never degraded service—and local repair is deterministic and auditable. *Learning displacement*: the RPS cap keeps output as minimal, explainable edits; deployments can surface failing tests before patches. Student programs come from the publicly released, de-identified Refactory dataset; no new human-subject data are introduced.

References

Ahmed, U. Z.; Fan, Z.; Yi, J.; Al-Bataineh, O. I.; and Roychoudhury, A. 2022. Verifix: Verified Repair of Programming Assignments. *ACM Transactions on Software Engineering and Methodology*, 31(4).

Anthropic. 2025. Prompt Caching — Claude API Documentation. https://docs.anthropic.com/en/docs/build-with-claude/prompt-caching. Cache reads are billed at 0.1x the base input-token price; accessed July 2026.

Cai, T.; Wang, X.; Ma, T.; Chen, X.; and Zhou, D. 2024. Large Language Models as Tool Makers. In *The Twelfth International Conference on Learning Representations, ICLR 2024*.

Chen, L.; Zaharia, M.; and Zou, J. 2024. FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance. *Transactions on Machine Learning Research*.

Dai, Z.; Chen, B.; Zhao, Z.; Tang, X.; Wu, S.; Yao, C.; Gao, Z.; and Chen, J. 2025. Less is More: Adaptive Program Repair with Bug Localization and Preference Learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*.

Gulwani, S.; Radicek, I.; and Zuleger, F. 2018. Automated Clustering and Program Repair for Introductory Programming Assignments. In *Proceedings of the 39th ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI 2018*, 465–480. ACM.

Hu, Y.; Ahmed, U. Z.; Mechtaev, S.; Leong, B.; and Roychoudhury, A. 2019. Re-Factoring Based Program Repair Applied to Programming Assignments. In *34th IEEE/ACM International Conference on Automated Software Engineering, ASE 2019*. IEEE.

Just, R.; Jalali, D.; Inozemtseva, L.; Ernst, M. D.; Holmes, R.; and Fraser, G. 2014. Are Mutants a Valid Substitute for Real Faults in Software Testing? In *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering, FSE 2014*. ACM.

Kim, D.; Nam, J.; Song, J.; and Kim, S. 2013. Automatic Patch Generation Learned from Human-Written Patches. In *Proceedings of the 35th International Conference on Software Engineering, ICSE 2013*. IEEE.

Liu, F.; Liu, Z.; Zhao, Q.; Jiang, J.; Zhang, L.; Li, G.; Sun, Z.; Li, Z.; and Ma, Y. 2024. FastFixer: An Efficient and Effective Approach for Repairing Programming Assignments. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering, ASE 2024*. ACM.

Liu, K.; Koyuncu, A.; Kim, D.; and Bissyandé, T. F. 2019. TBar: Revisiting Template-Based Automated Program Repair. In *Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA 2019*, 31–42. ACM.

Madaan, A.; Tandon, N.; Gupta, P.; Hallinan, S.; Gao, L.; Wiegreffe, S.; Alon, U.; Dziri, N.; Prabhumoye, S.; Yang, Y.; Gupta, S.; Majumder, B. P.; Hermann, K.; Welleck, S.; Yazdanbakhsh, A.; and Clark, P. 2023. Self-Refine: Iterative Refinement with Self-Feedback. In *Advances in Neural Information Processing Systems 36, NeurIPS 2023*.

OpenAI. 2024. Prompt Caching — OpenAI API Documentation. https://platform.openai.com/docs/guides/prompt-caching. Automatic prefix caching with discounted cached input tokens; see also https://openai.com/index/api-prompt-caching/; accessed July 2026.

Orvalho, P.; Janota, M.; and Manquinho, V. 2025. Counterexample Guided Program Repair Using Zero-Shot Learning and MaxSAT-based Fault Localization. In *Proceedings of the AAAI Conference on Artificial Intelligence*.

Shinn, N.; Cassano, F.; Gopinath, A.; Narasimhan, K.; and Yao, S. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. In *Advances in Neural Information Processing Systems 36, NeurIPS 2023*.

Wang, G.; Xie, Y.; Jiang, Y.; Mandlekar, A.; Xiao, C.; Zhu, Y.; Fan, L.; and Anandkumar, A. 2024. Voyager: An Open-Ended Embodied Agent with Large Language Models. *Transactions on Machine Learning Research*. ArXiv:2305.16291.

Wang, K.; Singh, R.; and Su, Z. 2018. Search, Align, and Repair: Data-Driven Feedback Generation for Introductory Programming Exercises. In *Proceedings of the 39th ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI 2018*. ACM.

Xia, C. S.; and Zhang, L. 2022. Less Training, More Repairing Please: Revisiting Automated Program Repair via Zero-Shot Learning. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ESEC/FSE 2022*. ACM.

Xia, C. S.; and Zhang, L. 2024. Automated Program Repair via Conversation: Fixing 162 out of 337 Bugs for $0.42 Each using ChatGPT. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA 2024*, 819–831. ACM.

Xiang, J.; Xu, X.; Kong, F.; Wu, M.; Zhang, Z.; Zhang, H.; and Zhang, Y. 2024. How Far Can We Go with Practical Function-Level Program Repair? arXiv preprint arXiv:2404.12833.

Yang, B.; Tian, H.; Pian, W.; Yu, H.; Wang, H.; Klein, J.; Bissyandé, T. F.; and Jin, S. 2024. CREF: An LLM-Based Conversational Software Repair Framework for Programming Tutors. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA 2024*, 882–894. ACM.

Yi, J.; Ahmed, U. Z.; Karkare, A.; Tan, S. H.; and Roychoudhury, A. 2017. A Feasibility Study of Using Automated Program Repair for Introductory Programming Assignments. In *Proceedings of the 11th Joint Meeting on Foundations of Software Engineering, ESEC/FSE 2017*, 740–751. ACM.

Zhang, J.; Cambronero, J.; Gulwani, S.; Le, V.; Piskac, R.; Soares, G.; and Verbruggen, G. 2022. Repairing Bugs in Python Assignments Using Large Language Models. arXiv preprint arXiv:2209.14876. Preprint; the archival version of this work appeared as PyDex at OOPSLA 2024 (Zhang et al. 2024).

Zhang, J.; Cambronero, J. P.; Gulwani, S.; Le, V.; Piskac, R.; Soares, G.; and Verbruggen, G. 2024. PyDex: Repairing

Bugs in Introductory Python Assignments using LLMs. *Proceedings of the ACM on Programming Languages*, 8(OOPSLA1).

Reproducibility Checklist

General paper structure. Conceptual outline of the approach: yes (Algorithm 1, Figure 1). Claims vs. speculation delineated: yes (RQ answer boxes state measured claims; the closing recipe is marked as argument). Pedagogical references for background: yes (Related Work).

Theoretical contributions. This paper makes no formal theoretical claims; the break-even expression in RQ2 is elementary algebra over measured quantities, with all symbols defined in the text.

Datasets. All experiments use the publicly available Refactory dataset (Hu et al. 2019); our seeded, scripted import pipeline (test parsing, per-test entry points, split sampling) is released, so the exact evaluation sets are reconstructible from the public corpus. No new human-subject data are collected.

Computational experiments. Source code, prompts, synthesized repairers, raw LLM responses, the ported Refactory-tool configuration, and analysis scripts are released; experiments run on CPU only. Model versions, pricing snapshot, tokenizer convention, decoding budget (single sample), split seed, timeouts, and the RPS threshold are stated in Experimental Setup and fixed in the released configuration. Metrics are formally defined; repair rates carry 95% Wilson intervals and paired claims use exact McNemar tests. Local search and evaluation are deterministic; per-submission LLM calls are single-shot by design (deployment matches one attempt per submission). Builder synthesis is stochastic; every round's prompt, response, and build-set outcome for both builder tiers is released, and the refinement loop's convergence behavior is reported in RQ3.